

Analysis: TF-IDF Question Answering System

This project implements a nearest neighbor approach to quiz bowl question answering using TF-IDF (Term Frequency-Inverse Document Frequency) vectorization. My implementation demonstrates several key insights about how text similarity works in practice for question-answering tasks.

What Gets Confused Most Often

Looking at the results, the system tends to confuse answers that share similar terminology or are from related domains. For example, when asked "This capital of England," my system incorrectly predicted "Maine" and "Boston" instead of "London." This suggests that geographical terms like state names are getting higher TF-IDF weights than country-specific terms, possibly because state-related questions appear more frequently in the training data. Similarly, for questions about authors or composers, the system shows strong performance on well-known figures like "Jane Austen" and "Wolfgang Amadeus Mozart," likely because these names appear frequently across multiple questions and receive distinctive TF-IDF weights. However, more obscure figures from similar time periods or genres tend to get confused with each other.

Comparison to GPT

The TF-IDF approach has clear limitations compared to GPT-based systems. While my TF-IDF guesser relies purely on semantic similarity between question texts, GPT's transformer architecture captures deeper contextual understanding and can handle more complex reasoning. The TF-IDF system essentially treats questions as "bags of words," missing syntactic structure and semantic nuances that GPT can leverage. However, the TF-IDF approach offers advantages in interpretability; I can easily examine which words are driving the similarity scores and understand why certain matches occur. The confidence scores are also more directly interpretable as cosine similarities between document vectors.

Recall Performance Analysis

My implementation shows varying recall performance as I increase the number of guesses. At 20% recall, I was correctly identifying answers for approximately 1 in 5 questions, which is reasonable for a baseline system. The 69% miss rate indicates that many questions require deeper reasoning than simple term similarity can provide. The remaining 12% categorized as "close" are particularly interesting, these represent cases where the TF-IDF similarity found semantically related content but missed the specific answer. This suggests that combining multiple guessers (like TF-IDF with GPT) could significantly improve performance by leveraging different strengths.

Technical Implementation Notes

I optimized my `batch_guess` function to handle multiple questions efficiently using NumPy's `argpartition` for faster top-k retrieval. The vectorizer uses English stop words removal and configurable min/max document frequency thresholds to filter vocabulary appropriately. The cosine similarity computation works well for sparse TF-IDF representations, and I ensured proper memory management to avoid creating dense matrices that would exceed available memory on the evaluation platform.

Conclusion

While TF-IDF provides a solid foundation for question similarity matching, its reliance on word frequency patterns limits its ability to handle complex reasoning tasks typical in quiz bowl. The system excels at finding questions with similar terminology but struggles with questions requiring deeper conceptual understanding or multi-hop reasoning. Future improvements could explore incorporating semantic embeddings or combining this with other retrieval methods to build more robust question-answering systems.